

Table 1: Comparison between LSTM, LSTM with attention, and our Ptr-Net model on the convex hull problem. Note that the baselines must be trained on the same n that they are tested on.

METHOD	TRAINED n	n	ACCURACY	AREA
LSTM [1]	50	50	1.9%	FAIL
+ATTENTION [5]	50	50	38.9%	99.7%
PTR-NET	50	50	72.6%	99.9%
LSTM [1]	5	5	87.7%	99.6%
PTR-NET	5-50	5	92.0%	99.6%
LSTM [1]	10	10	29.9%	FAIL
PTR-NET	5-50	10	87.0%	99.8%
PTR-NET	5-50	50	69.6%	99.9%
PTR-NET	5-50	100	50.3%	99.9%
PTR-NET	5-50	200	22.1%	99.9%
PTR-NET	5-50	500	1.3%	99.2%

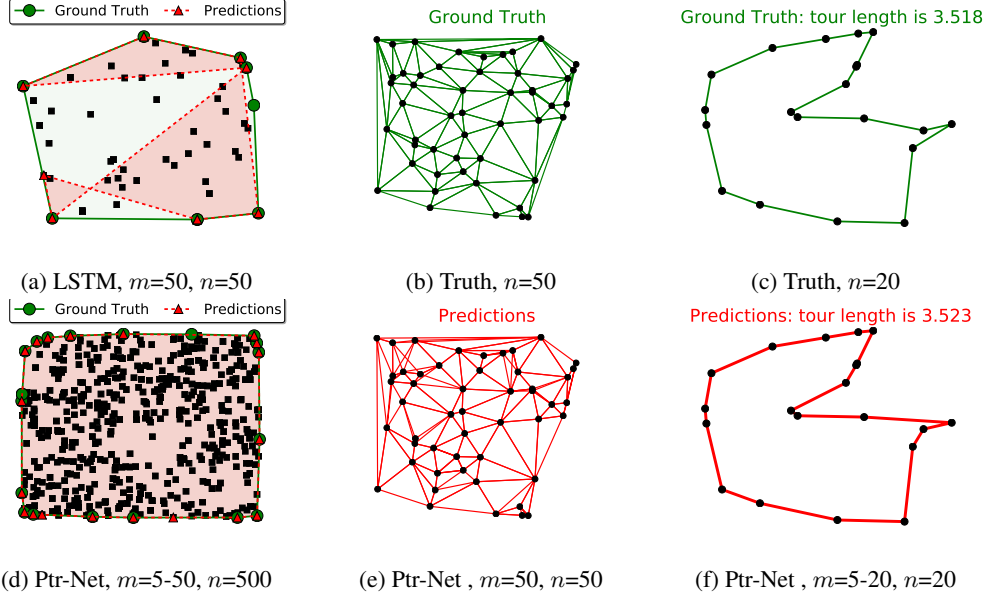


Figure 3: Examples of our model on Convex hulls (left), Delaunay (center) and TSP (right), trained on m points, and tested on n points. A failure of the LSTM sequence-to-sequence model for Convex hulls is shown in (a). Note that the baselines cannot be applied to a different length from training.

Using the Ptr-Net model for $n = 5$, we obtained an accuracy of 80.7% and triangle coverage of 93.0%. For $n = 10$, the accuracy was 22.6% and the triangle coverage 81.3%. For $n = 50$, we did not produce any precisely correct triangulation, but obtained 52.8% triangle coverage. See the middle column of Figure 3 for an example for $n = 50$.

4.4 Travelling Salesman Problem

We considered the planar symmetric travelling salesman problem (TSP), which is NP-hard as the third problem. Similarly to finding convex hulls, it also has sequential outputs. Given that the Ptr-Net implements an $O(n^2)$ algorithm, it was unclear if it would have enough capacity to learn a useful algorithm solely from data.

As discussed in Section 3.3, it is feasible to generate exact solutions for relatively small values of n to be used as training data. For larger n , due to the importance of TSP, good and efficient algorithms providing reasonable approximate solutions exist. We used three different algorithms in our experiments – A1, A2, and A3 (see Section 3.3 for references).

Table 2 shows all of our results on TSP. The number reported is the length of the proposed tour. Unlike the convex hull and Delaunay triangulation cases, where the decoder was unconstrained, in

Table 2: Tour length of the Ptr-Net and a collection of algorithms on a small scale TSP problem.

n	OPTIMAL	A1	A2	A3	PTR-NET
5	2.12	2.18	2.12	2.12	2.12
10	2.87	3.07	2.87	2.87	2.88
50 (A1 TRAINED)	N/A	6.46	5.84	5.79	6.42
50 (A3 TRAINED)	N/A	6.46	5.84	5.79	6.09
5 (5-20 TRAINED)	2.12	2.18	2.12	2.12	2.12
10 (5-20 TRAINED)	2.87	3.07	2.87	2.87	2.87
20 (5-20 TRAINED)	3.83	4.24	3.86	3.85	3.88
25 (5-20 TRAINED)	N/A	4.71	4.27	4.24	4.30
30 (5-20 TRAINED)	N/A	5.11	4.63	4.60	4.72
40 (5-20 TRAINED)	N/A	5.82	5.27	5.23	5.91
50 (5-20 TRAINED)	N/A	6.46	5.84	5.79	7.66

this example we set the beam search procedure to only consider valid tours. Otherwise, the Ptr-Net model would sometimes output an invalid tour – for instance, it would repeat two cities or decided to ignore a destination. This procedure was relevant for $n > 20$, where at least 10% of instances would not produce any valid tour.

The first group of rows in the table show the Ptr-Net trained on optimal data, except for $n = 50$, since that is not feasible computationally (we trained a separate model for each n). Interestingly, when using the worst algorithm (A1) data to train the Ptr-Net, our model outperforms the algorithm that is trying to imitate.

The second group of rows in the table show how the Ptr-Net trained on optimal data with 5 to 20 cities can generalize beyond that. The results are virtually perfect for $n = 25$, and good for $n = 30$, but it seems to break for 40 and beyond (still, the results are far better than chance). This contrasts with the convex hull case, where we were able to generalize by a factor of 10. However, the underlying algorithms are of far greater complexity than $O(n \log n)$, which could explain this phenomenon.

5 Conclusions

In this paper we described Ptr-Net, a new architecture that allows us to learn a conditional probability of one sequence $\mathcal{C}^{\mathcal{P}}$ given another sequence $\mathcal{P}$, where $\mathcal{C}^{\mathcal{P}}$ is a sequence of discrete tokens corresponding to positions in $\mathcal{P}$. We show that Ptr-Nets can be used to learn solutions to three different combinatorial optimization problems. Our method works on variable sized inputs (yielding variable sized output dictionaries), something the baseline models (sequence-to-sequence with or without attention) cannot do directly. Even more impressively, they outperform the baselines on fixed input size problems - to which both the models can be applied.

Previous methods such as RNNSearch, Memory Networks and Neural Turing Machines [5, 6, 2] have used attention mechanisms to process inputs. However these methods do not directly address problems that arise with variable output dictionaries. We have shown that an attention mechanism can be applied to the output to solve such problems. In so doing, we have opened up a new class of problems to which neural networks can be applied without artificial assumptions. In this paper, we have applied this extension to RNNSearch, but the methods are equally applicable to Memory Networks and Neural Turing Machines.

Future work will try and show its applicability to other problems such as sorting where the outputs are chosen from the inputs. We are also excited about the possibility of using this approach to other combinatorial optimization problems.

Acknowledgments

We would like to thank Rafal Jozefowicz, Ilya Sutskever, Quoc Le and Samy Bengio for useful discussions on this topic. We would also like to thank Daniel Gillick for his help with the final manuscript.

References

- [1] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [2] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [3] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning internal representations by error propagation. Technical report, DTIC Document, 1985.
- [4] Anthony J Robinson. An application of recurrent nets to phone probability estimation. *Neural Networks, IEEE Transactions on*, 5(2):298–305, 1994.
- [5] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR 2015, arXiv preprint arXiv:1409.0473*, 2014.
- [6] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *ICLEAR 2015, arXiv preprint arXiv:1410.3916*, 2014.
- [7] Oriol Vinyals, Lukasz Kaiser, Terry Koo, Slav Petrov, Ilya Sutskever, and Geoffrey Hinton. Grammar as a foreign language. *arXiv preprint arXiv:1412.7449*, 2014.
- [8] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. Show and tell: A neural image caption generator. In *CVPR 2015, arXiv preprint arXiv:1411.4555*, 2014.
- [9] Jeff Donahue, Lisa Anne Hendricks, Sergio Guadarrama, Marcus Rohrbach, Subhashini Venugopalan, Kate Saenko, and Trevor Darrell. Long-term recurrent convolutional networks for visual recognition and description. In *CVPR 2015, arXiv preprint arXiv:1411.4389*, 2014.
- [10] Wojciech Zaremba and Ilya Sutskever. Learning to execute. *arXiv preprint arXiv:1410.4615*, 2014.
- [11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [12] Nitish Srivastava, Elman Mansimov, and Ruslan Salakhutdinov. Unsupervised learning of video representations using lstms. In *ICML 2015, arXiv preprint arXiv:1502.04681*, 2015.
- [13] Ray A Jarvis. On the identification of the convex hull of a finite set of points in the plane. *Information Processing Letters*, 2(1):18–21, 1973.
- [14] Ronald L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Information processing letters*, 1(4):132–133, 1972.
- [15] Franco P. Preparata and Se June Hong. Convex hulls of finite sets of points in two and three dimensions. *Communications of the ACM*, 20(2):87–93, 1977.
- [16] S1 Rebay. Efficient unstructured mesh generation by means of delaunay triangulation and bowyer-watson algorithm. *Journal of computational physics*, 106(1):125–138, 1993.
- [17] Richard Bellman. Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM (JACM)*, 9(1):61–63, 1962.
- [18] Suboptimal travelling salesman problem (tsp) solver, 2015. Available at <https://github.com/dmishin/tsp-solver>.
- [19] Traveling salesman problem c++ implementation, 2015. Available at <https://github.com/samlbest/traveling-salesman>.
- [20] C++ implementation of traveling salesman problem using christofides and 2-opt, 2015. Available at <https://github.com/beckysag/traveling-salesman>.